

SensorTrustFusion

Descripción del Software

Plataforma científica para fusión multisensor adaptativa bajo degradación e incertidumbre de sensores

Autor: Francisco Javier Burgos Flórez

Versión: 1.0.0

Fecha: Septiembre de 2026

Contacto: fjburgosf@gmail.com

Titular de los derechos: Francisco Javier Burgos Flórez

Repositorio: <https://github.com/fjburgosf/SensorTrustFusion>

Software científico desarrollado y validado exclusivamente con simulación computacional y sensores virtuales; no requiere hardware externo.

Tabla de contenido

1. Información general del producto.....	4
2. Descripción general del software.....	4
2.1 Requisitos de hardware y software.....	5
2.2 Justificación de los requisitos.....	5
3. Fundamentos científicos y estado del arte.....	5
3.1 Contexto y problema científico.....	5
3.2 Estado del arte.....	6
3.3 Brecha y aporte metodológico.....	7
3.4 Formulación del método SensorTrust.....	7
3.5 Hipótesis que el software permite contrastar.....	8
3.6 Método experimental.....	8
4. Objetivos y área de aplicación.....	9
4.1 Objetivo general.....	9
4.2 Área de aplicación.....	9
5. Funcionalidades principales.....	9
6. Adaptabilidad, extensibilidad y mantenibilidad.....	10
7. Robustez, desempeño y consistencia.....	11
7.1 Robustez.....	11
7.2 Desempeño.....	11
7.3 Consistencia y reproducibilidad.....	11
8. Interfaz y usabilidad.....	12
9. Integridad y seguridad de la información.....	12
10. Portabilidad y compatibilidad.....	12
11. Documentación y soporte técnico.....	12
12. Verificación, validación y desempeño.....	13
13. Impacto y utilización.....	14
14. Contribución al estado del arte.....	14
14.1 Problema abordado.....	14

14.2 Soluciones relacionadas.....	15
14.3 Aporte.....	15
15. Aporte del autor.....	15
Referencias.....	15

1. Información general del producto

Tabla 1. Información general de SensorTrustFusion v1.0.0.

Campo	Información
Nombre del software	SensorTrustFusion
Subtítulo (inglés)	A Scientific Platform for Adaptive Multisensor Fusion Under Sensor Degradation and Uncertainty
Subtítulo (español)	Plataforma científica para fusión multisensor adaptativa bajo degradación e incertidumbre de sensores
Versión	1.0.0
Tipo de producto	Software científico de escritorio y biblioteca de Python para la investigación experimental en fusión multisensor, confiabilidad de sensores y degradación no estacionaria.
Campo / área	Fusión sensorial, estimación de estado, filtrado de Kalman, detección y diagnóstico de fallos, confiabilidad de sensores, instrumentación, sistemas dinámicos, robótica y navegación, métodos de Monte Carlo y análisis de sensibilidad.
Autor	Francisco Javier Burgos Flórez (autor único)
Contacto	fjburgosf@gmail.com
Titular de los derechos	Francisco Javier Burgos Flórez
Año de desarrollo	2026
Estado del software	Funcional; probado en Linux y Windows; ejecutable autocontenido para Windows.
Lenguaje de programación	Python 3.10 o superior; interfaz gráfica en PySide6 (Qt).
Plataformas	Windows (ejecutable y código fuente), Linux y macOS (código fuente).
Hardware asociado	Ninguno. Basta un computador convencional; no se necesitan sensores, tarjetas de adquisición ni instrumentación.
Repositorio de código fuente	https://github.com/fjburgosf/SensorTrustFusion

2. Descripción general del software

SensorTrustFusion es un software científico para diseñar, simular, analizar y repetir experimentos de fusión multisensor en condiciones en las que uno o varios sensores se degradan con el tiempo. Su núcleo es la estimación en línea de la confiabilidad $T_i(k) \in [0, 1]$ de cada sensor a partir de los dos primeros momentos de la innovación, y el estudio del efecto que esa confiabilidad tiene sobre la calidad de la estimación.

El problema que motivó su desarrollo es bien conocido por quien ha intentado evaluar un algoritmo de fusión adaptativa. Para saber si el algoritmo funciona hace falta conocer el valor verdadero de la variable medida, decidir cuándo aparece un fallo, de qué tipo es y qué tan grande, correr varios algoritmos sobre exactamente los mismos datos y poder repetir el experimento tantas veces como sea necesario. Con sensores físicos casi nada de esto es posible. Por eso SensorTrustFusion trabaja con una cadena totalmente simulada, en la que cada etapa alimenta a la siguiente:

1. un simulador genera la evolución verdadera del sistema dinámico;
2. los sensores virtuales producen mediciones con ruido, incertidumbre y las degradaciones configuradas;
3. los algoritmos de fusión y de estimación procesan esas mediciones y, en el caso de SensorTrust, estiman la confianza en cada sensor;
4. con la estimación resultante se calculan métricas científicas y, si se desea, estadística sobre muchas corridas;
5. todo queda guardado de manera que el experimento pueda reproducirse.

En la práctica, el programa genera la trayectoria verdadera de un modelo dinámico y las mediciones de tantos sensores virtuales como se quiera, cada uno con su propio ruido y sus propias degradaciones, y conserva el valor verdadero de cada fallo. Sobre ese **mismo** conjunto de datos ejecuta métodos de fusión estáticos, filtros de Kalman (KF, EKF y UKF) con distintas estrategias para la covarianza y el método SensorTrust. Después calcula métricas de estimación, consistencia, detección, confianza, pesos y robustez. También puede lanzar estudios Monte Carlo, barridos de parámetros, envolventes de robustez y análisis de sensibilidad. Cada resultado se guarda junto con su configuración, su semilla y huellas criptográficas que permiten comprobar más tarde que se reprodujo exactamente.

Carácter científico. Lo que hace científico a este software no es que use ecuaciones, sino lo que permite hacer con ellas. Con SensorTrustFusion se diseñan experimentos con valor verdadero conocido, se modelan sensores y degradaciones, se desarrollan y comparan algoritmos, se contrastan hipótesis con estadística multi-corrida y se mide la sensibilidad de los resultados a cada parámetro, siempre con experimentos repetibles gracias a las semillas. Aunque el método SensorTrust es su aporte central, la plataforma sirve para otras preguntas de investigación: cuánto aporta el segundo momento frente a la media, cómo se comporta la confianza cuando el sensor se recupera, qué severidad de fallo tolera cada método o cuándo conviene un Kalman adaptativo en lugar de una fusión basada en confianza. La sección 3 presenta los fundamentos, el estado del arte y las hipótesis que el software permite contrastar.

Usuarios. Está pensado para investigadores, estudiantes de posgrado y personal técnico que trabajen en estimación, fusión sensorial, instrumentación, control, robótica o navegación.

2.1 Requisitos de hardware y software

Tabla 2. Requisitos de hardware y software.

Requisito	Detalle
Computador	Computador convencional con Windows 10/11 de 64 bits, Linux o macOS; se recomiendan 4 GB de RAM o más para ejecutar Monte Carlo en paralelo.
Ejecutable (Windows)	SensorTrustFusion-1.0.0-win64.exe, un único archivo que no requiere instalar Python.
Código fuente	Python 3.10 o superior, con NumPy, SciPy, pandas, Matplotlib, PyYAML, SALib y PySide6.
Hardware externo	No se requiere.
Red	No se requiere; una vez instaladas las dependencias, todo funciona sin conexión.

2.2 Justificación de los requisitos

Se eligió Python porque su ecosistema científico (NumPy, SciPy, pandas y Matplotlib) ofrece álgebra lineal, estadística y visualización maduras y de uso libre. SALib aporta implementaciones de referencia de los métodos de sensibilidad de Morris y Sobol, y PySide6 permite una interfaz de escritorio que funciona igual en los tres sistemas operativos. Prescindir del hardware fue una decisión deliberada: solo en simulación se conoce el valor verdadero y se controlan por completo las degradaciones, y eso es lo que hace que los experimentos sean objetivos, repetibles y comparables. Para quien no quiera instalar Python, el ejecutable de Windows se compila con PyInstaller en un flujo de integración continua.

3. Fundamentos científicos y estado del arte

3.1 Contexto y problema científico

La fusión multisensor combina las mediciones de varias fuentes para obtener una estimación más exacta y confiable que la de cualquier sensor por separado. En sistemas dinámicos, el marco

probabilístico más usado es el filtro de Kalman con sus extensiones [1], [2]. Las revisiones del área coinciden en que el reto central es la imperfección de los datos: incertidumbre, sesgos, valores atípicos e información en conflicto [3]. Trabajos recientes sobre vehículos autónomos [4], [5] y manufactura inteligente [6] muestran que el problema aparece en muchos campos, porque la seguridad de esos sistemas depende de cómo se comportan los sensores en conjunto, y ese comportamiento cambia con el envejecimiento, el ambiente, los fallos intermitentes o la pérdida de calibración.

El filtro de Kalman y sus versiones no lineales (EKF y UKF [7]) suponen que la covarianza del ruido de medición R_i de cada sensor se conoce y no cambia. Cuando un sensor se degrada de forma **no estacionaria** (aparece o crece un sesgo, aumenta la varianza, el sensor deriva o se congela), esa suposición deja de cumplirse y la estimación fusionada empeora, aunque los demás sensores sigan funcionando bien. La pregunta que aborda SensorTrustFusion es la siguiente:

¿Cómo estimar en línea la confiabilidad de cada sensor a partir de la evidencia estadística que ya ofrece el filtro (la innovación), de modo que la fusión reduzca el peso de los sensores degradados, y cómo medir de forma objetiva y reproducible cuánto se gana con ello frente a las alternativas del estado del arte?

3.2 Estado del arte

Filtrado adaptativo por covarianza. Una primera familia de métodos estima en línea las estadísticas del ruido. Los enfoques clásicos son el estimador de Sage–Husa [8] y los métodos de Mehra basados en la correlación y el emparejamiento de covarianza de la innovación [9], que luego se aplicaron a la navegación integrada [10]. Los métodos bayesianos variacionales estiman a la vez el estado y las covarianzas [11], y más adelante se extendieron al caso en que tanto la covarianza de proceso como la de medición son inexactas [12]. Entre los trabajos recientes, [13] estima ambas covarianzas con suavizado de Kalman y [14] añade detección de puntos de cambio para reaccionar ante saltos bruscos del ruido. Todos estos métodos ajustan **la magnitud de la covarianza**. El problema es que un sesgo no es ruido de media cero: un estimador de covarianza lo toma como varianza adicional y, en general, no distingue un sesgo de un aumento del ruido.

Detección y diagnóstico de fallos basados en residuos. En un filtro consistente la innovación es blanca, de media cero y de covarianza conocida. De ahí que las pruebas chi-cuadrado sobre residuos, o sobre diferencias entre estimaciones, sean desde hace décadas una herramienta estándar para detectar fallos en filtros de Kalman [15]. Las revisiones sobre diagnóstico y tolerancia a fallos distinguen entre enfoques basados en modelos, en señales y en datos [16], y la revisión sistemática de [17] sobre sensores y sistemas de monitoreo señala como retos abiertos la detección de fallos incipientes y la validación comparativa de los métodos. Entre los esquemas recientes de fusión tolerante a fallos están los filtros de múltiples modelos interactuantes aplicados a robots móviles [18], y los trabajos de pronóstico [19] insisten en que la degradación suele ser progresiva. Una compuerta chi-cuadrado sin memoria decide muestra por muestra, así que le cuesta detectar las degradaciones lentas.

Fusión con correlaciones desconocidas. Cuando se fusionan estimaciones cuya correlación cruzada se desconoce, la intersección de covarianzas (CI) garantiza que el resultado siga siendo consistente [20]. Un cuarto de siglo después, la revisión de [21] muestra que las correlaciones desconocidas siguen siendo un problema abierto. SensorTrustFusion incluye CI como arquitectura de fusión a nivel de estimaciones, de manera que la confianza también puede estudiarse en ese esquema.

Plataformas de software. Existen marcos abiertos de seguimiento y estimación como Stone Soup [22], con componentes, simuladores y métricas; bibliotecas de filtros como FilterPy [23], y productos comerciales como el Sensor Fusion and Tracking Toolbox [24]. Al revisar su

documentación no se encontró un entorno que reuniera a la vez cuatro elementos: degradaciones no estacionarias con perfiles temporales y valor verdadero de los fallos, estimación de confianza por sensor, comparación de métodos sobre exactamente las mismas mediciones, y estudios Monte Carlo, de robustez y de sensibilidad global con trazabilidad criptográfica.

3.3 Brecha y aporte metodológico

De lo anterior se desprenden dos brechas.

1. **Brecha metodológica.** Los métodos adaptativos de covarianza responden a la magnitud total de la innovación, y las compuertas chi-cuadrado, a muestras aisladas. Ninguno separa de forma explícita la evidencia de **sesgo** (primer momento) de la evidencia de **dispersión** (segundo momento central), ni la acumula con una memoria estadística controlada.
2. **Brecha experimental.** Para evaluar un método adaptativo bajo degradación se necesita el valor verdadero, control sobre el instante, el tipo, el perfil y la magnitud de los fallos, repetición exacta y estadística sobre muchas corridas. Nada de esto se consigue con sensores físicos, y construir esa infraestructura para cada estudio es costoso y dificulta comparar un trabajo con otro.

SensorTrustFusion atiende las dos. Por un lado implementa un estimador de confianza basado en los dos primeros momentos de la innovación; por otro, ofrece una plataforma experimental completa, reproducible y sin hardware, en la que ese estimador y las alternativas del estado del arte se evalúan en igualdad de condiciones.

3.4 Formulación del método SensorTrust

Para cada sensor i , la innovación respecto al estado a priori se estandariza con la covarianza **nominal**:

$$\varepsilon_{i,k} = L_{i,k}^{-1} (z_{i,k} - \hat{z}_{i,k \vee k-1}), S_{i,k} = H_i P_{k \vee k-1} H_i^T + R_i = L_{i,k} L_{i,k}^T. \quad (1)$$

Si el sensor funciona bien (hipótesis nominal H_0), $E[\varepsilon_{i,k}] = 0$ y $E[\varepsilon_{i,k} \varepsilon_{i,k}^T] = I$. Un sesgo desplaza el primer momento, mientras que un aumento del ruido eleva el segundo momento sin mover la media. Los momentos se estiman en línea (EWMA, ventana deslizante o media recursiva) con un número efectivo de muestras N_{eff} :

$$\hat{\mu}_{i,k} = \lambda \hat{\mu}_{i,k-1} + (1 - \lambda) \varepsilon_{i,k}, \hat{m}_{2,i,k} = \lambda \hat{m}_{2,i,k-1} + (1 - \lambda) \frac{\varepsilon_{i,k}^T \varepsilon_{i,k}}{m}, \hat{s}_{i,k}^2 = \hat{m}_{2,i,k} - \frac{\|\hat{\mu}_{i,k}\|^2}{m}. \quad (2)$$

Lo que excede ciertas zonas muertas estadísticas, expresadas en desviaciones estándar de los estimadores bajo H_0 , se convierte en pérdida de confianza:

$$d_{1,i} = \max(0, \|\hat{\mu}_i^c\| / \sqrt{m} - \gamma_1 / \sqrt{N_{eff}}), d_{2,i} = \max(0, \hat{s}_i^2 - 1 - \gamma_2 \sqrt{2/N_{eff}}), \quad (3)$$

$$\tilde{T}_i = \rho_i^0 \exp(-d_{1,i}/\beta_1 - d_{2,i}/\beta_2), T_i(k) = T_i(k-1) + \eta[\tilde{T}_i - T_i(k-1)]. \quad (4)$$

La tasa η es asimétrica: la confianza se pierde rápido y se recupera despacio. El término $\hat{\mu}_i^c$ es la media medida respecto a la mediana de los sensores redundantes que siguen siendo confiables (la referencia de consenso), y es lo que evita culpar al sensor equivocado cuando el que falla es precisamente el que domina la estimación. La confianza entra en la fusión inflando la covarianza de medición:

$$R_i^{eff} = \frac{R_i}{\max\left(T_{min}, T_i / \max_j T_j\right)}. \quad (5)$$

Así, un filtro de Kalman, un EKF, un UKF o una fusión por intersección de covarianzas se convierten en su versión «SensorTrust» sin tocar su estructura. El Manual Técnico desarrolla la formulación completa, junto con la compuerta de innovación y el detector de degradación, que aplica pruebas chi-cuadrado sobre los mismos momentos.

3.5 Hipótesis que el software permite contrastar

Tabla 3. Hipótesis que el software permite contrastar y diseño experimental asociado.

Hipótesis	Diseño experimental en SensorTrustFusion
H1. La confianza basada en los dos primeros momentos reduce el error de estimación frente al KF nominal cuando un sensor sufre sesgo, deriva o congelamiento.	Benchmarks ST-BENCH-02 a 12 con los mismos datos para todos los métodos; Monte Carlo con pruebas pareadas de Wilcoxon y corrección de Holm.
H2. El segundo momento central aporta información que el primero no contiene (aumento de varianza sin sesgo).	Ablaciones «solo primer momento» y «solo segundo momento»; Monte Carlo de ablación con muestreo por hipercubo latino.
H3. La referencia de consenso evita atribuir el fallo al sensor equivocado cuando el degradado es el más preciso.	ST-BENCH-16 con y sin consenso.
H4. Para cada método existe una severidad de degradación a partir de la cual deja de cumplir una especificación.	Envoltentes y mapas de robustez, y barridos paramétricos.
H5. El desempeño depende más de unos parámetros (severidad, ruido, escala de confianza) que de otros.	Análisis de sensibilidad global de Morris y de Sobol [25]–[28].

3.6 Método experimental

La forma de experimentar de la plataforma sigue prácticas bien establecidas. El valor verdadero se genera aparte y nunca llega a los estimadores; los métodos «oráculo», que sí usan información verdadera (la R real o el mejor sensor), están marcados como tales y solo sirven de cota de referencia.

La comparación entre métodos es justa por construcción: se genera un único conjunto de datos por escenario y todos los métodos trabajan sobre él. Además, cada experimento incluye un «gemelo nominal», que comparte exactamente el mismo ruido pero sin degradación, para medir cuánto desempeño se pierde por culpa del fallo.

En los estudios estadísticos se usan semillas derivadas y diseños aleatorios, por hipercubo latino o de Sobol; los resultados se resumen con intervalos de confianza del 95 % y los métodos se comparan con la prueba pareada de rangos con signo de Wilcoxon [29] y la corrección de Holm para comparaciones múltiples [30]. La sensibilidad global se estudia con los métodos de Morris [26] y de índices de Sobol [27], siguiendo [25] e implementados con SALib [28].

Para la reproducibilidad, la configuración se normaliza, cada proceso aleatorio tiene su propia semilla y cada experimento guarda un manifiesto con huellas SHA-256 que luego se verifica, en línea con las reglas de reproducibilidad computacional de [31] y con los principios FAIR para software de investigación [32]. La base numérica son NumPy [33] y SciPy [34]. Las figuras usan

paletas aptas para daltonismo y combinan estilos de línea y marcadores para no depender solo del color [35].

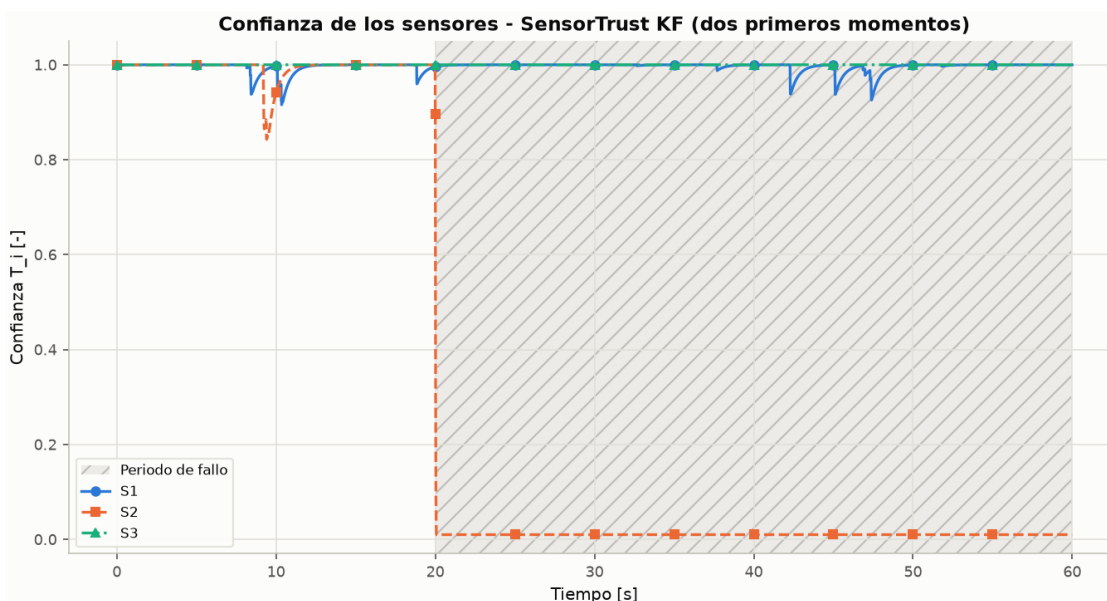


Figura 1. Confianza estimada por SensorTrust en ST-BENCH-02 (sesgo abrupto en el sensor S2; franja rayada: intervalo del fallo)

En este experimento de referencia, la confianza del sensor degradado S2 cae por debajo de 0,5 a los 0.02 s de iniciado el fallo y durante él promedia 0.011. Los sensores sanos mantienen una confianza mínima de 0.916 (S1) y 1.000 (S3).

4. Objetivos y área de aplicación

4.1 Objetivo general

Ofrecer una plataforma computacional reproducible para modelar, simular, analizar y comparar algoritmos de fusión multisensor en condiciones nominales y bajo incertidumbre, fallos y degradación no estacionaria, que permita estimar en línea la confiabilidad de las mediciones y cuantificar su efecto sobre la calidad de la estimación.

4.2 Área de aplicación

El software es útil en fusión multisensor y estimación de estado, en la detección, el diagnóstico y la tolerancia a fallos de sensores, y en la confiabilidad y autovalidación de instrumentación. Sus casos de uso naturales están en robótica, vehículos autónomos y navegación, y en el monitoreo de procesos térmicos o mecánicos con sensores redundantes. También sirve para la docencia y la investigación en filtrado de Kalman, métodos de Monte Carlo y análisis de sensibilidad.

5. Funcionalidades principales

Entradas. Un experimento se configura desde la interfaz gráfica, con un archivo YAML/JSON o desde Python. La configuración incluye el modelo dinámico y sus parámetros, el estado inicial y su incertidumbre, la duración y la frecuencia de muestreo, y los sensores con todas sus características (qué miden, frecuencia, ruido, sesgo, resolución, rango, retardo, pérdida de datos y confiabilidad nominal). También se definen las degradaciones con sus perfiles temporales, los

métodos de fusión, los parámetros de confianza, las métricas, la semilla y, si se desea, estudios de varias corridas. De manera opcional pueden importarse datos externos en CSV, TXT o JSON.

Procesamiento. El programa incluye seis modelos dinámicos (señal escalar, velocidad constante, aceleración constante, oscilador masa–resorte–amortiguador, sistema térmico de dos nodos y péndulo no lineal) y admite modelos lineales definidos por el usuario. El valor verdadero lo produce un generador independiente al que los estimadores no tienen acceso.

Se puede simular cualquier número de sensores virtuales, con ocho modelos de ruido: gaussiano, uniforme, variante en el tiempo, dependiente del tiempo, heterocedástico, correlacionado, en ráfagas e impulsivo. El motor de degradación ofrece quince tipos de fallo, entre ellos el sesgo constante y progresivo, la deriva, la varianza creciente, los valores atípicos, la saturación, el congelamiento y la pérdida de muestras, y ocho perfiles temporales (escalón, rampa, exponencial, sigmoide, sinusoidal, caminata aleatoria, intermitente y recuperación) que pueden combinarse entre sí.

En la parte de estimación hay filtros KF, EKF y UKF numéricamente estables, fusión centralizada y fusión a nivel de estimaciones por intersección de covarianzas, y cuatro estrategias para la covarianza de medición: nominal, verdadera (oráculo), adaptativa de Sage–Husa y SensorTrust. SensorTrust estandariza la innovación, estima sus dos primeros momentos (con EWMA, ventana o media recursiva), usa una referencia de consenso entre sensores redundantes y ajusta la confianza con una dinámica asimétrica; se complementa con una compuerta de innovación y con un detector de degradación independiente.

Las métricas cubren el error de estimación (RMSE, MAE, MSE, NRMSE, error máximo, sesgo, varianza del error, error final y RMSE por fase), la consistencia (NIS y NEES), la detección (retardo, verdaderos y falsos positivos, falsos negativos, precisión, exhaustividad, F1 y tasas de falsa alarma y de detección fallida), la confianza y los pesos, y el comportamiento global (éxito, divergencia, pérdida de desempeño y calidad de la recuperación). Por último, el programa ejecuta estudios Monte Carlo (con muestreo aleatorio, hipercubo latino o Sobol), barridos paramétricos, envolventes y mapas de robustez y análisis de sensibilidad OAT, Morris y Sobol, y resume los resultados con estadística descriptiva, intervalos de confianza y pruebas pareadas de Wilcoxon con corrección de Holm.

Salidas. Cada experimento crea una carpeta con un identificador consecutivo (ST_EXP_2026_000001, ...). Allí se guardan la configuración, el manifiesto de reproducibilidad, el conjunto de datos, las estimaciones, la confianza, las innovaciones, las métricas en JSON y CSV, doce tipos de figuras (PNG, SVG y PDF) y el registro de la ejecución. Los estudios de varias corridas guardan además sus tablas y figuras: diagramas de caja, funciones de distribución empírica, histogramas, mapas de calor, mapas de robustez y gráficos de sensibilidad.

El Manual de Usuario explica cómo operar el programa, y el Manual Técnico, los modelos y las ecuaciones.

6. Adaptabilidad, extensibilidad y mantenibilidad

El código está organizado en catorce paquetes con responsabilidades bien delimitadas: `models`, `simulation`, `sensors`, `degradation`, `estimators`, `trust`, `fusion`, `metrics`, `statistics`, `experiments`, `visualization`, `io`, `gui` y `utils`. La interfaz gráfica no contiene lógica científica; todas sus acciones se delegan en el gestor de experimentos y en los motores de estudio.

Para extender el programa existen interfaces explícitas (DynamicModel, VirtualSensor, NoiseModel, Degradation, Profile, GaussianFilter, TrustEstimator, Detector y FusionMethod) y registros en los que se inscriben los componentes nuevos. Así se pueden añadir modelos, ruidos, degradaciones, estimadores de confianza o métodos de fusión sin modificar el resto del sistema. El estimador de los dos primeros momentos, en particular, está separado de los filtros, los modelos, la simulación y la interfaz, y por eso puede combinarse con KF, EKF, UKF o con la fusión a nivel de estimaciones; de esa combinación salen SensorTrust KF, EKF, UKF y CI.

Cada experimento es un archivo YAML/JSON legible, y cualquier parámetro puede variarse en un estudio indicando su ruta (por ejemplo, `sensors[1].degradations[0].severity`). En cuanto al mantenimiento, la arquitectura está organizada por capas, el ejecutable de Windows se compila en integración continua, hay un registro de eventos (logging) y los errores se informan con mensajes comprensibles.

7. Robustez, desempeño y consistencia

7.1 Robustez

Antes de ejecutar cualquier experimento, el programa valida la configuración: tipos y rangos de los valores, dimensiones de las matrices, simetría y semidefinición de las covarianzas, número y nombres de los sensores, distribuciones, semillas, que los instantes de fallo estén dentro del horizonte simulado y que los archivos no estén dañados. Si algo falla, lo explica con un mensaje claro tanto en la interfaz como en la línea de comandos.

En lo numérico, la ganancia de Kalman se obtiene resolviendo sistemas lineales en lugar de invertir matrices, la covarianza se actualiza con la forma de Joseph y se simetriza, se usan factorizaciones robustas y la confianza y las covarianzas tienen límites inferiores. Si un método no se puede aplicar a un escenario o falla, el error se registra y los demás métodos siguen corriendo; la divergencia numérica también se detecta y se informa. Los cálculos largos se ejecutan en segundo plano y pueden cancelarse.

7.2 Desempeño

La tabla siguiente reúne los tiempos medidos en las ejecuciones de referencia, hechas en un computador con Linux y 4 núcleos.

Tabla 4. Tiempos de ejecución medidos en las ejecuciones de referencia.

Métrica	Resultado
SensorTrust KF, 6000 muestras, 3 sensores	1.25 s
KF nominal, 6000 muestras, 3 sensores	0.60 s
Experimento completo ST-BENCH-02 (7 métodos, gemelo nominal, figuras y archivos)	8.6 s
Monte Carlo de 100 corridas × 5 métodos (40 s a 100 Hz), en paralelo	62.3 s

7.3 Consistencia y reproducibilidad

Cada proceso aleatorio usa un generador derivado de la semilla y de una clave propia, de modo que la misma configuración con la misma semilla produce exactamente las mismas mediciones y los mismos resultados; esto se comprobó durante el desarrollo. Cada experimento guarda su configuración normalizada y un manifiesto con la versión, la semilla y las huellas SHA-256 de la configuración y del conjunto de datos. La orden `sensortrust verify` vuelve a generar el

escenario y compara las huellas. La comparación entre métodos es justa desde el diseño, porque todos los algoritmos trabajan sobre el mismo conjunto de datos.

8. Interfaz y usabilidad

La interfaz de escritorio está en español y en inglés, y se cambia de idioma con un botón. Tiene once pestañas: Sistema, Sensores, Degradación (con una vista previa calculada por el mismo motor de simulación), Fusión, Confianza, Experimento, Monte Carlo, Análisis (robustez, sensibilidad y barridos), Resultados (figuras por método y por sensor, y tablas de métricas), Comparar y Exportar (figuras, configuración, datos, métricas, verificación de reproducibilidad e importación opcional de datos).

Todos los parámetros aparecen con un nombre descriptivo, su símbolo y su unidad. Al cambiar de idioma cambian también los nombres de las métricas, las figuras y los mensajes de error. Cada pestaña lleva un recuadro que explica para qué sirve, cada control muestra una ayuda al pasar el ratón y el menú Ayuda ofrece una guía rápida y acceso a los manuales. Los doce ejemplos científicos y los benchmarks se cargan desde la barra de herramientas. En las figuras se usan colores aptos para daltonismo, estilos de línea, marcadores, leyendas, unidades y franjas rayadas para marcar los fallos, de modo que se entienden incluso impresas en escala de grises.

9. Integridad y seguridad de la información

SensorTrustFusion es una aplicación local: no ofrece servicios de red, no maneja usuarios ni credenciales y no transmite datos. Por esa razón la autenticación y el cifrado no aplican. Sí aplican otros controles. Las configuraciones y los archivos importados se validan antes de usarse. La integridad y la trazabilidad se garantizan con manifiestos que llevan huellas SHA-256, con la configuración normalizada, con identificadores consecutivos que se reservan de forma atómica y con el registro de cada ejecución. Por último, el valor verdadero está separado de los estimadores, que solo reciben las mediciones, y los métodos oráculo están identificados como tales.

10. Portabilidad y compatibilidad

Tabla 5. Compatibilidad por plataforma.

Plataforma	Compatibilidad
Windows	Ejecutable autocontenido y código fuente; verificado en Windows.
Linux	Código fuente; verificado en Linux, donde se hicieron las ejecuciones de referencia y las capturas.
macOS	Debería funcionar, porque todas las dependencias son multiplataforma, pero no se ha verificado.

La capa científica está escrita en Python sobre NumPy y SciPy y no depende de la plataforma; solo la interfaz depende de PySide6.

11. Documentación y soporte técnico

Tabla 6. Documentación y recursos disponibles.

Recurso	Disponible	Descripción
Manual de Usuario	Sí (carpeta Manuales/ del repositorio; incluido en el ejecutable)	Instalación, ejecución, interfaz, configuración, guía paso a paso, experimentos A–L, Monte Carlo, comparación, exportación y solución de problemas.

Recurso	Disponible	Descripción
Manual Técnico	Sí (carpeta Manuales/ del repositorio; incluido en el ejecutable)	Fundamentos, modelos, ecuaciones, algoritmos, métricas, arquitectura, formatos, reproducibilidad, verificación, validación y limitaciones.
README	Sí	Descripción, instalación, ejecución, estructura y benchmarks.
Código fuente	Sí	Repositorio del proyecto y archivo comprimido del código fuente en la sección Releases.
Ejecutable	Sí	SensorTrustFusion-1.0.0-win64.exe (sección Releases del repositorio).
Configuraciones y benchmarks	Sí	Carpeta configs/, benchmarks/ y examples/.

12. Verificación, validación y desempeño

Durante el desarrollo se verificó cada parte del programa frente a resultados conocidos: los modelos, los sensores y el ruido; la degradación, comprobando que no tenga ningún efecto antes del instante de fallo y que el perfil se aplique exactamente; el filtro de Kalman, comparándolo con la ecuación de Riccati y con soluciones analíticas; la innovación, los momentos, la confianza y los pesos; las métricas, las semillas y la reproducibilidad; la exportación e importación de datos; los estudios de varias corridas, y la interfaz gráfica. El detalle está en el Manual Técnico [36], sección 25.

La tabla siguiente resume algunos resultados de las ejecuciones de referencia (RMSE de posición; la especificación es 0,05 m).

Tabla 7. RMSE de posición [m] de tres métodos en escenarios representativos.

Escenario	KF (R nominal)	KF adaptativo	SensorTrust KF
Nominal (ST-BENCH-01)	0.0206	0.0207	0.0207
Sesgo abrupto (ST-BENCH-02)	0.2174	0.0241	0.0224
Sesgo progresivo (ST-BENCH-03)	0.2510	0.0256	0.0227
Sensor congelado (ST-BENCH-07)	0.6473	0.0256	0.0233
Sensor dominante con sesgo progresivo (ST-BENCH-16)	0.3463	0.4462	0.0215

En el estudio Monte Carlo de referencia se hicieron 100 corridas con un sesgo abrupto cuyo instante, magnitud y ruido se sortearon en cada corrida. Las tasas de éxito obtenidas fueron las siguientes.

Tabla 8. Tasa de éxito por método en el estudio Monte Carlo de sesgo abrupto.

Método	Tasa de éxito (RMSE $\leq$ 0,05 m)	Tasa de divergencia	Corridas
Ponderación por inverso de varianza	0.00	0.00	100
Filtro de Kalman (R nominal)	0.14	0.00	100
KF adaptativo (R Sage-Husa)	1.00	0.00	100
KF con compuerta NIS	1.00	0.00	100
SensorTrust KF (dos primeros momentos)	1.00	0.00	100

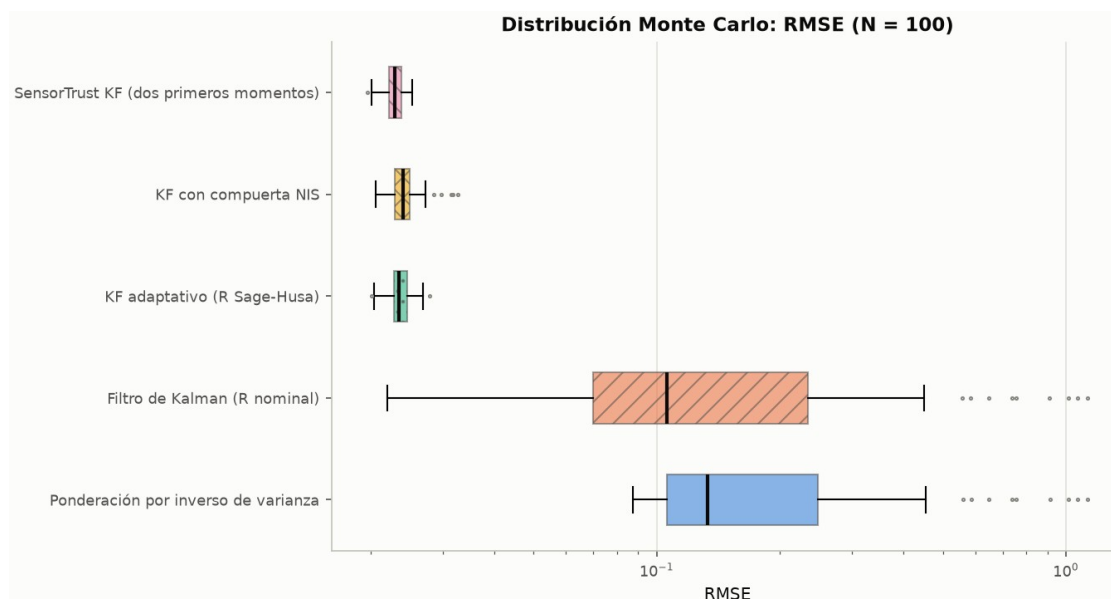


Figura 2. Distribución del RMSE en el estudio Monte Carlo de referencia (sesgo abrupto con instante, magnitud y ruido aleatorios); la línea de la especificación es 0,05 m

Las tablas completas, la validación del filtro, la envolvente de robustez y el análisis de sensibilidad están en el capítulo 26 del Manual Técnico.

13. Impacto y utilización

Conviene separar el impacto que el software podría tener, por lo que es capaz de hacer, de la evidencia real de uso. La versión 1.0.0 es una plataforma funcional y validada, pero a la fecha de este documento todavía no hay evidencia de utilización efectiva, es decir, proyectos, publicaciones o cursos que la hayan empleado. Lo que sigue describe, entonces, su potencial.

Investigación. Ofrece un entorno reproducible para estudiar métodos adaptativos de fusión bajo degradación, con valor verdadero conocido y comparación justa. Los resultados de referencia ya permiten sacar conclusiones cuantitativas. Por ejemplo, muestran que una confianza basada solo en innovaciones individuales culpa al sensor equivocado cuando el degradado domina la fusión, y que la referencia de consenso de SensorTrust corrige ese caso (ST-BENCH-16).

Docencia. Permite practicar filtrado de Kalman, consistencia (NIS/NEES), detección de fallos, Monte Carlo y análisis de sensibilidad sin laboratorio ni equipos.

Transferencia. Con el modelo lineal personalizado y la importación de datos externos se pueden probar estrategias de confiabilidad de sensores para una aplicación concreta antes de implementarlas.

14. Contribución al estado del arte

14.1 Problema abordado

Los algoritmos de filtrado de Kalman, de filtrado adaptativo por emparejamiento de covarianza y de intersección de covarianzas están bien documentados. Lo que no es sencillo es evaluarlos bajo degradación no estacionaria, porque para eso se necesita una infraestructura experimental: valor verdadero, degradaciones con perfiles temporales controlados y etiquetas de fallo, comparación

sobre datos idénticos, métricas de confianza y de detección, estudios Monte Carlo, de robustez y de sensibilidad, y reproducibilidad.

14.2 Soluciones relacionadas

Tabla 9. Comparación de SensorTrustFusion con soluciones relacionadas.

Solución	Enfoque	Simulación de degradaciones con valor verdadero	Estimación de confianza por sensor	Estudios Monte Carlo, robustez y sensibilidad integrados	Interfaz gráfica
FilterPy [23]	biblioteca de filtros en Python	no identificado	no identificado	no	no
Stone Soup [22]	marco abierto de seguimiento y estimación con simuladores y métricas	no identificado como foco	no identificado	parcial (métricas)	no identificado
MATLAB Sensor Fusion and Tracking Toolbox [24]	producto comercial de fusión y seguimiento	no identificado como foco	no identificado	no identificado	aplicaciones específicas
SensorTrustFusion (este trabajo)	plataforma experimental de fusión bajo degradación	sí, 15 tipos y 8 perfiles	sí (dos primeros momentos, consenso)	sí	sí, bilingüe

Esta comparación se basa únicamente en la documentación consultada. Cuando una celda dice «no identificado» significa que la capacidad no aparece en la fuente, no necesariamente que la herramienta carezca de ella.

14.3 Aporte

SensorTrustFusion reúne en una sola plataforma abierta y reproducible cuatro elementos que normalmente están dispersos: la simulación de degradaciones no estacionarias con el valor verdadero de los fallos, la estimación de confianza basada en los dos primeros momentos de la innovación con referencia de consenso, la comparación justa de métodos (con los oráculos claramente identificados) y los estudios Monte Carlo, de robustez y de sensibilidad. Y todo ello sin necesidad de hardware.

15. Aporte del autor

Francisco Javier Burgos Flórez es el único autor de SensorTrustFusion v1.0.0. Estuvieron a su cargo la concepción del proyecto, la formulación científica, el diseño y la implementación del software, su validación y la documentación.

Referencias

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] Y. Bar-Shalom, X. R. Li y T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York: Wiley, 2001.

- [3] B. Khaleghi, A. Khamis, F. O. Karray y S. N. Razavi, "Multisensor data fusion: A review of the state-of-the-art," *Information Fusion*, vol. 14, no. 1, pp. 28–44, 2013, doi: 10.1016/j.inffus.2011.08.001.
- [4] D. J. Yeong, G. Velasco-Hernandez, J. Barry y J. Walsh, "Sensor and sensor fusion technology in autonomous vehicles: A review," *Sensors*, vol. 21, no. 6, art. 2140, 2021, doi: 10.3390/s21062140.
- [5] J. Fayyad, M. A. Jaradat, D. Gruyer y H. Najjaran, "Deep learning sensor fusion for autonomous vehicle perception and localization: A review," *Sensors*, vol. 20, no. 15, art. 4220, 2020, doi: 10.3390/s20154220.
- [6] A. Tsanousa, E. Bektsis, C. Kyriakopoulos, A. Gómez González, U. Leturiondo, I. Gialampoukidis, A. Karakostas, S. Vrochidis e I. Kompatsiaris, "A review of multisensor data fusion solutions in smart manufacturing: Systems and trends," *Sensors*, vol. 22, no. 5, art. 1734, 2022, doi: 10.3390/s22051734.
- [7] S. J. Julier y J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004, doi: 10.1109/JPROC.2003.823141.
- [8] A. P. Sage y G. W. Husa, "Adaptive filtering with unknown prior statistics," en *Proc. Joint Automatic Control Conference*, 1969, pp. 760–769.
- [9] R. K. Mehra, "Approaches to adaptive filtering," *IEEE Transactions on Automatic Control*, vol. 17, no. 5, pp. 693–698, 1972, doi: 10.1109/TAC.1972.1100100.
- [10] A. H. Mohamed y K. P. Schwarz, "Adaptive Kalman filtering for INS/GPS," *Journal of Geodesy*, vol. 73, no. 4, pp. 193–203, 1999, doi: 10.1007/s001900050236.
- [11] S. Särkkä y A. Nummenmaa, "Recursive noise adaptive Kalman filtering by variational Bayesian approximations," *IEEE Transactions on Automatic Control*, vol. 54, no. 3, pp. 596–600, 2009, doi: 10.1109/TAC.2008.2008348.
- [12] Y. Huang, Y. Zhang, Z. Wu, N. Li y J. Chambers, "A novel adaptive Kalman filter with inaccurate process and measurement noise covariance matrices," *IEEE Transactions on Automatic Control*, vol. 63, no. 2, pp. 594–601, 2018, doi: 10.1109/TAC.2017.2730480.
- [13] T. Kruse et al., "Adaptive Kalman filtering: Measurement and process noise covariance estimation using Kalman smoothing," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3528348.
- [14] X. Hou, S. Zhao, J. Hu y H. Lan, "Noise-adaptive state estimators with change-point detection," *Sensors*, vol. 24, no. 14, art. 4585, 2024, doi: 10.3390/s24144585.
- [15] B. Brumback y M. Srinath, "A chi-square test for fault-detection in Kalman filters," *IEEE Transactions on Automatic Control*, vol. 32, no. 6, pp. 552–554, 1987, doi: 10.1109/TAC.1987.1104658.
- [16] Z. Gao, C. Cecati y S. X. Ding, "A survey of fault diagnosis and fault-tolerant techniques—Part I: Fault diagnosis with model-based and signal-based approaches," *IEEE Transactions on Industrial Electronics*, vol. 62, no. 6, pp. 3757–3767, 2015, doi: 10.1109/TIE.2015.2417501.
- [17] N. Trapani y L. Longo, "Fault detection and diagnosis methods for sensors systems: A scientific literature review," *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 1253–1263, 2023, doi: 10.1016/j.ifacol.2023.10.1749.

- [18] M. Kheirandish, E. A. Yazdi, H. Mohammadi y M. Mohammadi, "A fault-tolerant sensor fusion in mobile robots using multiple model Kalman filters," *Robotics and Autonomous Systems*, vol. 161, 2023.
- [19] M. Kordestani, M. Saif, M. E. Orchard, R. Razavi-Far y K. Khorasani, "Failure prognosis and applications—A survey of recent literature," *IEEE Transactions on Reliability*, vol. 70, no. 2, pp. 728–748, 2021, doi: 10.1109/TR.2019.2930195.
- [20] S. J. Julier y J. K. Uhlmann, "A non-divergent estimation algorithm in the presence of unknown correlations," en *Proc. American Control Conference*, 1997, pp. 2369–2373.
- [21] R. Forsling, B. Noack y G. Hendeby, "A quarter century of covariance intersection: Correlations still unknown?," *IEEE Control Systems Magazine*, vol. 44, no. 2, pp. 81–105, 2024, doi: 10.1109/MCS.2024.3358658.
- [22] J. Barr et al., "Stone Soup open source framework for tracking and state estimation: Enhancements and applications," en *Proc. SPIE 12122, Signal Processing, Sensor/Information Fusion, and Target Recognition XXXI*, art. 1212205, 2022, doi: 10.1117/12.2618495.
- [23] R. R. Labbe, *FilterPy: Kalman filters and other optimal and non-optimal estimation filters in Python* (software). Disponible: <https://github.com/rlabbe/filterpy>
- [24] The MathWorks, Inc., *Sensor Fusion and Tracking Toolbox* (software). Disponible: <https://www.mathworks.com/products/sensor-fusion-and-tracking.html>
- [25] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana y S. Tarantola, *Global Sensitivity Analysis: The Primer*. Chichester: Wiley, 2008.
- [26] M. D. Morris, "Factorial sampling plans for preliminary computational experiments," *Technometrics*, vol. 33, no. 2, pp. 161–174, 1991.
- [27] I. M. Sobol', "Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates," *Mathematics and Computers in Simulation*, vol. 55, no. 1–3, pp. 271–280, 2001.
- [28] T. Iwanaga, W. Usher y J. Herman, "Toward SALib 2.0: Advancing the accessibility and interpretability of global sensitivity analyses," *Socio-Environmental Systems Modelling*, vol. 4, art. 18155, 2022, doi: 10.18174/sesmo.18155.
- [29] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [30] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [31] G. K. Sandve, A. Nekrutenko, J. Taylor y E. Hovig, "Ten simple rules for reproducible computational research," *PLoS Computational Biology*, vol. 9, no. 10, art. e1003285, 2013, doi: 10.1371/journal.pcbi.1003285.
- [32] M. Barker, N. P. Chue Hong, D. S. Katz, A.-L. Lamprecht, C. Martinez-Ortiz, F. Psomopoulos et al., "Introducing the FAIR principles for research software," *Scientific Data*, vol. 9, art. 622, 2022, doi: 10.1038/s41597-022-01710-x.
- [33] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.
- [34] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[35] F. Crameri, G. E. Shephard y P. J. Heron, “The misuse of colour in science communication,” *Nature Communications*, vol. 11, art. 5444, 2020, doi: 10.1038/s41467-020-19160-7.

[36] SensorTrustFusion v1.0.0, *Manual de Usuario* y *Manual Técnico* (documentos que acompañan esta descripción).